

B.Sc. (Data Science)

DS101T : Problem Solving and Python Programming

Unit 2: Introduction to Python

Mrs. Reshma Masurekar
Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18

Unit 2: Introduction to Python

- 1.1 History, feature of Python, setting up path, working with python Interpreter, basic syntax, variable and data types, operators
- 1.2 Conditional statements-If, If-Else, nested if-else, Examples.
- 1.3 Looping-For, While, Nested loops, Examples
- 1.4 Control Statements-Break, Continue, Pass.
- 1.5 String Manipulation-Accessing String, Basic Operations, String Slices, Function and Methods, Examples.
- 1.6 Lists-Introduction, accessing list, operations, working with lists, function & methods.
- 1.7 Tuple-Introduction, accessing tuples, operations working, function & methods, Examples.
- 1.8 Dictionaries-Introduction, Accessing values in dictionaries, working with dictionaries, properties, function, Examples.
- 1.9 Functions-Defining a function, Calling a function, types of function, function arguments, anonymous function, global & local variable, Example



History



- Python laid its foundation in the late 1980s.
- Python language implementation was started in December 1989 by **Guido Van Rossum** at Centrum Wiskunde & Informatica in Netherland.
- In February 1991, **Guido Van Rossum** published the code labeled version 0.9.0.
- Python is High level , Dynamic and General Purpose programming language.
- The following programming languages influence Python:
 - ABC language.
 - Modula-3

Why python called python?

- **Guido van Rossum** was reading the script of a popular BBC comedy series "**Monty Python's Flying Circus**". It was late on-air 1970s.
- Van Rossum wanted to select a name which unique, sort, and little-bit mysterious. So he decided to select naming Python after the "**Monty Python's Flying Circus**" for their newly created programming language.
- The **comedy series** was creative and well random. It talks about everything. Thus it is slow and unpredictable, which made it very interesting.
- Python is also versatile and widely used in every technical field, such as Machine Learning, Artificial Intelligence, Web Development, Mobile Application Desktop Application, Scientific Calculation, etc.

Python

- Python is basically a **high-level programming language**
- It is a **dynamic and free open-source language** in nature.
- Moreover, it uses an **interpreter** for converting the **source code into machine code**.
- Furthermore, it supports both **object-oriented programming as well as procedure-oriented programming**.
- It is such a language that is highly readable and uses English keywords.

- In **February 1991**, published labeled version 0.9.0
- In **1994**, Python 1.0 was released with new features like exception handling, lambda, map, filter, and reduce.
- **2000**, Python 2.0 List comprehensions, Garbage collection system
- **3rd December 2008**, 3.0

Features of Python

- Simple and Easy to use
- Platform Independent
- Free and Open source
- Interpreted Language
- Rich Standard Library
- Object-oriented
- Supports GUI programming
- Integrated and Extensible in nature
- Portable
- Dynamically Typed
- Scalable

Applications of Python

- Data Science
- Artificial Intelligence
- Machine Learning
- Games
- Hardware/Sensor
- Robotics
- Mobile Applications
- Gaming
- Internet of Things
- Desktop Applications
- Data Analysis and Preprocessing

Python – Basic

- The Python syntax defines a set of rules that are used to create a Python Program.
- The Python Programming Language Syntax has many similarities to Perl, C, and Java Programming Languages. However, there are some definite differences between the languages.
- A [Python program to print "Hello, World!"](#) in two different modes of Python Programming.
 - (a) Interactive Mode Programming
 - (b) Script Mode Programming.

Python - Interactive Mode Programming

 Python 3.11 (64-bit)

```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec 6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello")
Hello
>>>
```

- Here >>> denotes a Python Command Prompt to type user commands and press the Enter

Python - Script Mode Programming

- We can invoke the Python interpreter with a script parameter which begins the execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.
- Python files have extension **.py**.
- **Execute the file with command:** Python filename.py
-

Python Identifiers

- A Python identifier is a name used to identify a variable, function, class, module or other object.
- An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).
- Python does not allow punctuation characters such as @, \$, and % within identifiers.
- Python is a case sensitive programming language.
- **Example:** Number and number are two different identifiers in Python.

Python Reserved Words

- Are Python keywords.
- Python keywords are unique words reserved with defined meanings and functions that we can only apply for those functions.
- Reserved words cannot be used as constant or variable or any other identifier names.
- All the Python keywords contain lowercase letters only.
- Python contains thirty-five keywords

Python Reserved Words

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Comments in Python

- **Comments in Python** are the lines in the code that are ignored by the interpreter during the execution of the program.
- A developer uses comments to write note or remark or thought process in the source code. It explains the basic logic behind why a particular line of code was written.
- Some significant uses include:
 - Increasing readability
 - Explaining the code to others
 - Understanding the code easily after a long-term
 - Logic Explanation
 - Including resources
 - Re-using the existing code

Types of Comments

- Single-line comment
- Multi-Line Comment

Single-Line Comments

- Single-line comments **begin with the “#” character.**
- Anything that is written in a single line after ‘#’ is considered as a comment.
- The syntax:
 # Add comment here

Multi-Line Comments

- Python does not provide the facility for multi-line comments.
- **With Multiple Hashtags (#):** To add a multiline comment you could insert a # for each line

- **Example:**

```
# it is a
```

```
# comment
```

- **Using String Literals:** Since Python will ignore string literals that are not assigned to a variable, you can add a multiline string (triple quotes) in your code, and place your comment inside it:

- **Example**

```
"""      This is a comment  
        written in more than just one line """
```

Variables

- A variable is the name given to a memory location.
- Variables are containers for storing data values.
- A value-holding Python variable is also known as an identifier.
- **Rules:**
 1. Variable name start with a letter or **underscore character**.
 2. Variable name cannot start with a number.
 3. Variable name can only contain alpha numeric characters and underscores(A-Z, a-z, 0-9, or underscore).
 4. White space and special characters (!, @, #, %, etc.) are not allowed in the identifier name. ^, &, *).
 5. Names of identifiers are case-sensitive.
- Examples of **valid** identifiers: a123, _n, n_9, etc.
- Examples of **invalid** identifiers: 1a, n%4, n 9, etc.

Declaring Variable and Assigning Values

- Python has **no command to declare a variable**.
- Python allows **to create a variable at the necessary time**.
- In Python, we don't have to explicitly declare variables. The variable is declared automatically whenever a value is added to it.
- The equal (=) operator is utilized to assign worth to a variable.
- **Example:**

```
a = 10  
b = "John"  
print(a)  
print(b)
```
- Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

- **Assigning values to Variables in Python**

Example:

```
a,b,c=5,10,15
```

```
print a
```

```
print b
```

```
print c
```

- **Assigning values to Variables in Python**

Example:

```
name = "Devansh"
```

```
age = 20
```

```
marks = 80.50
```

```
print(name)
```

```
print(age)
```

```
print(marks)
```

- String variables can be declared either by using single or double quotes.

Example:

```
str1 = "Data"
```

```
print(str1)
```

#double quotes are the same as single quotes:

```
str2 = 'Science'
```

```
print(str2)
```

Output:

Data

Science

type()

- *type()* function is used to get the data type of a variable.

Example:

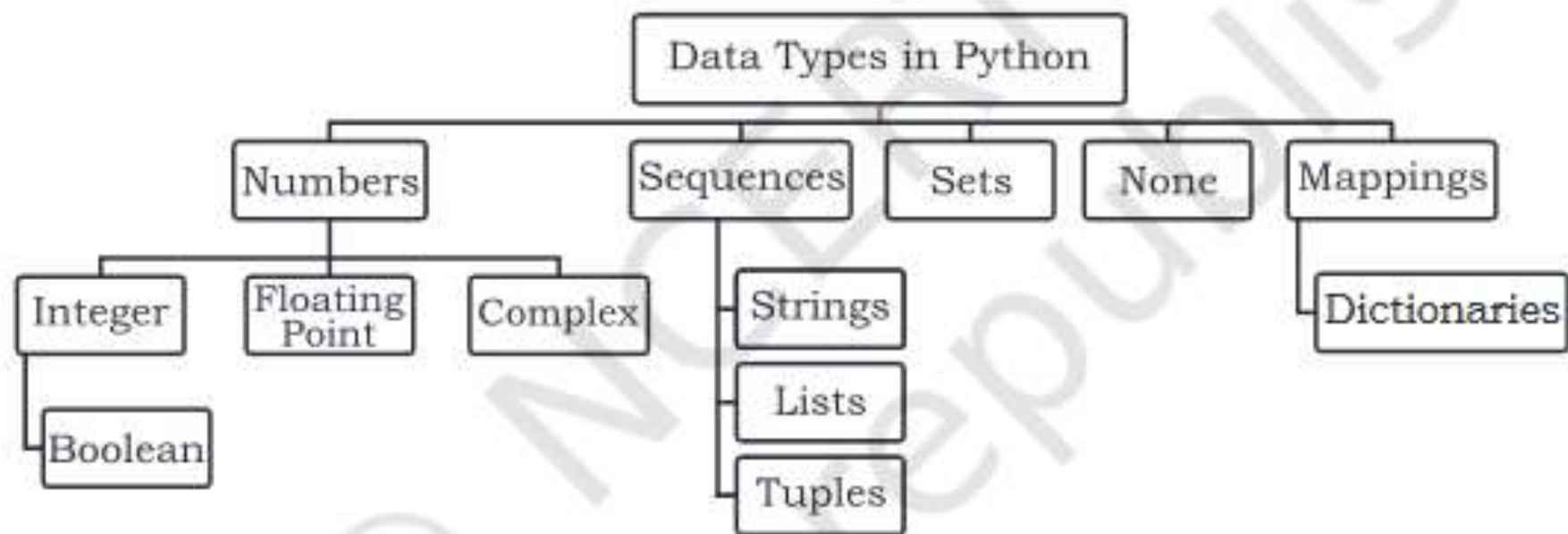
```
a = 5
b = "Data Science"
c = 10.5
print(type(a))
print(type(b))
print(type(c))
```

Output:

```
<class 'int'>
<class 'str'>
<class 'float'>
```

Python Data Types

- Every value has a datatype, and variables can hold values.
- The following is a list of the Python-defined data types.
 - Numeric Types: int, float, complex
 - Text Type: str
 - Boolean Type :bool
 - Sequence Types: list, tuple, range
 - Mapping Type: dict
 - Set Types :set
 - Binary Types: bytes, bytearray
 - None Type: None



Numeric Types: int, float, complex

- **int, or integer**, is a whole number, positive or negative, without decimals, of unlimited length.
- **Float**, or "floating point number" is a number, positive or negative, containing one or more decimals.
- **Complex numbers** A complex number contains the real and imaginary parts. i.e., $a + ib$.
- **Example:**

int:

x = 10

y =

35656333445889911

z = -3166333

Float

x = 4.50

y = 5.0

z = -15.28

Complex

x = 5+4j

y = 7j

z = -5j

Text Type: str

- In Python, a **string is a sequence of characters**.
- Strings in python are surrounded by either single quotation marks, or double quotation marks.
- 'Science' is the same as "Science".

- **Example:**

```
str = "Science"  
print(str)
```

```
str = "Data"+"Science"  
print(str)
```

- operator + is used to combine two strings.
- **Example:** "Data"+" Science" returns "Data Science"

Boolean: bool

- True and False are the two default values for the Boolean type.
- These qualities are utilized to decide the given assertion valid or misleading.
- False can be represented by the 0 or the letter "F," while true can be represented by any value that is not zero.
- **Example:**
Python program to check the boolean type
print(type(True))
print(type(False))
- **Output:**
<class 'bool'>
<class 'bool'>

List

- Lists in Python are like arrays in C, but lists can contain data of different types.
- A list is created by placing elements inside square brackets [], separated by commas.
- **Ordered** - They maintain the order of elements.
- **Mutable** - Items can be changed after creation.
- **Allow duplicates** - They can contain duplicate values.
- **Example:**

```
# a list of three elements
```

```
ages = [19, 26, 29]
```

```
print(ages)
```

```
# Output: [19, 26, 29]
```

Tuple

- A Tuple is like a list.
- Tuples, also contain a collection of items from various data types.
- **Ordered** - They maintain the order of elements.
- **Immutable** - They cannot be changed after creation.
- **Allow duplicates** - They can contain duplicate values.
- A tuple is created by placing elements inside a round **brackets ()** separated by commas.
- **Example:**
 `numbers = (1, 2, -5)`
 `print(numbers)`
 # Output: (1, 2, -5)

range

- The [range](#) type represents an immutable sequence of numbers and is commonly used for looping a specific number of times in [for](#) loops.

- **Syntax:**

`range(stop)`

`range(start, stop)`

`range(start, stop[, step])`

Note: If the *step* argument is omitted, it defaults to 1. and If the *start* argument is omitted, it defaults to 0

- **Example:**

`list(range(10))` `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

`list(range(1, 11))` `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`

`list(range(0))` `[]`

Mapping Type: Dictionary

- A Python dictionary is a collection of items, similar to lists and tuples.
- A dictionary is a **key-value pair** set **arranged in any order**.
- It stores a specific value for each key.
- a dictionary is created by placing key: value pairs inside curly **brackets {}**, separated by commas.
- **Example:**

```
d = {1:'Arun', 2:'Varun', 3:'Ram', 4:'Raj'}  
print (d)
```
- **Output:**

```
{1:'Arun', 2:'Varun', 3:'Ram', 4:'Raj'}
```


Set

- A Set is a **unordered** collection of **unique** items enclosed within the curly braces{, }.
- A set is a collection of **unique** data, meaning that elements within a set **cannot be duplicated**.
- **Unordered** collection: The elements of a set have no set order; Set items can appear in a different order every time while using it.
- **Mutable** (can change after creation)
- In Python, set is created by placing all the elements inside curly braces {}, separated by commas.
- **Example:**

```
set1={'C','C++','Java','Python','Java'}  
print(set1)
```

Binary types

- In Python, bytes and bytearray are used to work with binary data and memory views of binary data.
- The **bytes** is an **immutable** sequence type to represent sequences of bytes (8-bit values). Each element in a bytes object is an integer in the range [0, 255].
- Bytes are defined using the b prefix followed by a sequence of bytes enclosed in single or double quotes.
- Example: `byte1 = b'Data Science'`
- The **bytearray** is similar to **bytes** but bytearray objects can be modified after creation.
- Example: `bytearray1= bytearray([72, 101, 108, 108, 111])`

- A memoryview type to create a “view” of memory containing binary data. It doesn’t store the data itself but provides a view into the memory where the data is stored.

- Example

```
data = bytearray([1, 2, 3, 4, 5])
```

```
mem_view = memoryview(data)
```

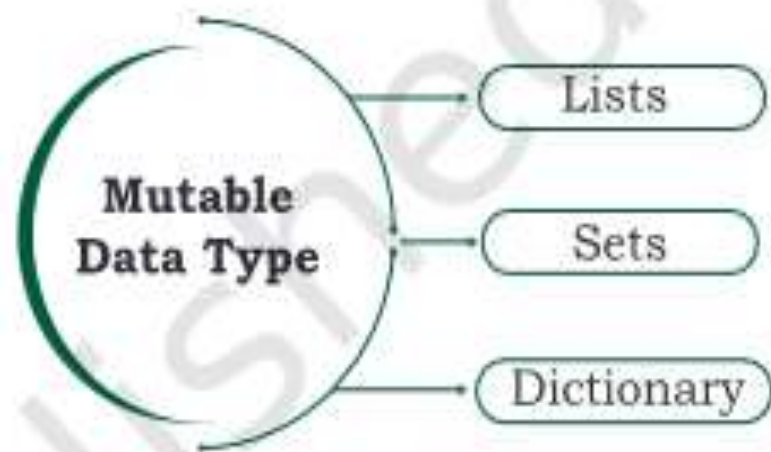
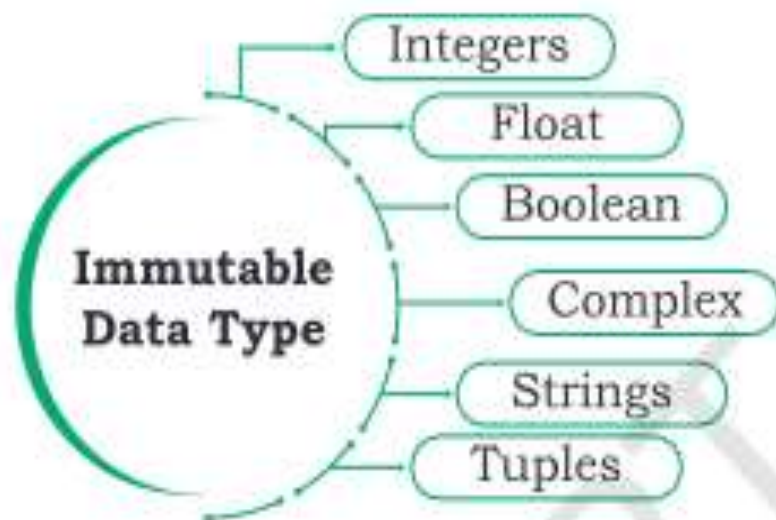
None

- The ***None*** represents a special value indicating the absence of a value.
- Example: `no_value = None`

- Mutable data types in Python
 - List
 - Set
 - Dictionary
- Immutable data types in Python
 - Number
 - String
 - Tuple

Mutable and Immutable Data Types

- Variables whose values can be changed after they are created and assigned are called **mutable**.
- Variables whose values cannot be changed after they are created and assigned are called **immutable**.
- When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in memory.



String	List	Tuple	Set	Dictionary
Immutable	Mutable	Immutable	Mutable	Mutable
Ordered/Indexed	Ordered/Indexed	Ordered/Indexed	Unordered	Unordered
Allows Duplicate Members	Allows Duplicate Members	Allows Duplicate Members	Doesn't allow Duplicate Members	Doesn't allow Duplicate Members
Empty String=""	Empty list=[]	Empty tuple=()	Empty Set=set()	Empty dictionary={ }
String with single element="H"	List with single item=["Hello"]	Tuple with single item=("Hello")	Set with single item={"Hello"}	Dictionary with single item={"Hello":1}
It can store data type string only.	It can store any data type str, list, set, tuple, int, dic	It can store any data type str, list, set, tuple,	It can store data types (int, str, tuple) but not	Inside of dictionary key can be int, str and tuple o can be

Type Casting

- Type casting is used to the data type of a variable from one type to another type.

#convert from float to int:

```
b = int(2.8)
```

#convert from int to float:

```
a = float(1)
```

#convert from int to complex:

```
c = complex(1)
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

```
print(type(a))
```

Output:

```
1.0
```

```
2
```

```
(1+0j)
```

```
<class 'float'>
```

```
<class 'int'>
```

```
<class 'complex'>
```

Random number generation

- Python has a built-in module called random that can be used to make random numbers:
- Import the random module, and display a random number between 1 and 9:
- ```
import random
print(random.randrange(1, 10))
```

# Input()

- Allows user input.
- Syntax: `input(prompt)`
- Prompt specifies a String, representing a default message before the input.

```
name=input("Enter Name:")
```

```
print(name)
```

**Output:** Ram

```
num1=input("Enter First Number:")
```

```
num2=input("Enter Second Number:")
```

```
print(num1+num2)
```

**Ouput:**

Enter First Number:12

Enter Second Number:13

1213

# Escape characters

- Are special or nonprintable characters in a string
  - `\n` : Newline
  - `\t` : Tab
  - `\` : Backslash
  - `\r` : Carriage return

# print()

- The **print()** function **prints** the specified message to the screen, or other standard output device.
- The message can be a string, or any other object, the object will be converted into a string before written to the screen.
- **Syntax:** `print(value(s), sep= ' ', end = '\n', file=file, flush=flush)`
  - **value(s):** Any value, and as many as you like. Will be converted to a string before printed
  - **sep='separator' : (Optional)** Specify how to separate the objects, if there is more than one. Default : ' '
  - **end='end': (Optional)** Specify what to print at the end. Default : '\n'
  - **file : (Optional)** An object with a write method. Default : sys.stdout
  - **flush : (Optional)** A Boolean, specifying if the output is flushed (True) or buffered (False). Default: False
  - **Return Type:** It returns output to the screen.

# Quotes

- `Single('....')`: used to print one line statement
- `Double("....")`: used to print one line statement
- `Triple("""....""")`: used to print multiline statement
- String is enclosed either with single quotes or double quotes.
- `print('Data Science')`      `print("Data Science")`
- `Print("""Dr. D. Y. Patil ACS College,  
Pimpri  
Pune 18""")`

# Operator

- The operator is a symbol that performs a specific operation between two operands.
- Python divides the operators in the following groups:
  - Arithmetic operators
  - Assignment operators
  - Comparison operators
  - Logical operators
  - Identity operators
  - Membership operators
  - Bitwise operators



| Operator Type        | Operators                                                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Arithmetic operators | +, -, *, /, %, **(Exponent)                                                                                                        |
| Assignment operators | =, +=, -=, *=, /=                                                                                                                  |
| Relational operators | ==, !=, <, >, <=, >=,                                                                                                              |
| Logical operators    | AND, OR, NOT                                                                                                                       |
| Bitwise operators    | &,  , ^, ~, <<, >><br>bitwise AND (&), bitwise OR ( ),<br>bitwise XOR (^), negation (~), Left<br>shift (<<), and Right shift (>>). |
| Identity operators   | is, is not                                                                                                                         |
| Membership operators | in, not in                                                                                                                         |

# Arithmetic operators:

used with numeric values to perform common mathematical operations

| Operator | Name           | Example  |
|----------|----------------|----------|
| +        | Addition       | $x + y$  |
| -        | Subtraction    | $x - y$  |
| *        | Multiplication | $x * y$  |
| /        | Division       | $x / y$  |
| %        | Modulus        | $x \% y$ |
| **       | Exponentiation | $x ** y$ |
| //       | Floor division | $x // y$ |

# Assignment Operators:

used to assign values to variables

| Operator | Example       | Same As        |
|----------|---------------|----------------|
| =        | x = 5         | x = 5          |
| +=       | x += 3        | x = x + 3      |
| -=       | x -= 3        | x = x - 3      |
| *=       | x *= 3        | x = x * 3      |
| /=       | x /= 3        | x = x / 3      |
| %=       | x %= 3        | x = x % 3      |
| //=      | x //= 3       | x = x // 3     |
| **=      | x **= 3       | x = x ** 3     |
| &=       | x &= 3        | x = x & 3      |
| =        | x  = 3        | x = x   3      |
| ^=       | x ^= 3        | x = x ^ 3      |
| >>=      | x >>= 3       | x = x >> 3     |
| <<=      | x <<= 3       | x = x << 3     |
| :=       | print(x := 3) | x = 3 print(x) |

# Relational Operators:

## used to compare two values

| Operator | Name                     | Example |
|----------|--------------------------|---------|
| ==       | Equal                    | x == y  |
| !=       | Not equal                | x != y  |
| >        | Greater than             | x > y   |
| <        | Less than                | x < y   |
| >=       | Greater than or equal to | x >= y  |
| <=       | Less than or equal to    | x <= y  |
| <=       | Less than or equal to    | x <= y  |

# Logical operators:

used to combine conditional statements

| Operator | Description                                             | Example                                 |
|----------|---------------------------------------------------------|-----------------------------------------|
| and      | Returns True if both statements are true                | $x < 5$ and $x < 10$                    |
| or       | Returns True if one of the statements is true           | $x < 5$ or $x < 4$                      |
| not      | Reverse the result, returns False if the result is true | $\text{not}(x < 5 \text{ and } x < 10)$ |

# Identity Operators:

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location

| Operator | Description                                            | Example    |
|----------|--------------------------------------------------------|------------|
| is       | Returns True if both variables are the same object     | x is y     |
| is not   | Returns True if both variables are not the same object | x is not y |

# Membership Operators:

used to test if a sequence is presented in an object

| Operator | Description                                                                      | Example    |
|----------|----------------------------------------------------------------------------------|------------|
| in       | Returns True if a sequence with the specified value is present in the object     | x in y     |
| not in   | Returns True if a sequence with the specified value is not present in the object | x not in y |

# Bitwise Operators:

used to compare (binary) numbers

| Operator | Name                 | Description                                                                                             | Example      |
|----------|----------------------|---------------------------------------------------------------------------------------------------------|--------------|
| &        | AND                  | Sets each bit to 1 if both bits are 1                                                                   | $x \& y$     |
|          | OR                   | Sets each bit to 1 if one of two bits is 1                                                              | $x   y$      |
| ^        | XOR                  | Sets each bit to 1 if only one of two bits is 1                                                         | $x \wedge y$ |
| ~        | NOT                  | Inverts all the bits                                                                                    | $\sim x$     |
| <<       | Zero fill left shift | Shift left by pushing zeros in from the right and let the leftmost bits fall off                        | $x << 2$     |
| >>       | Signed right shift   | Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off | $x >> 2$     |



# Operator precedence:

- Describes the order in which operations are performed.
- If two operators have the same precedence, the expression is evaluated from left to right.

The precedence order is described in the table below, starting with the highest precedence at the top

| Operator | Description                                           |
|----------|-------------------------------------------------------|
| ()       | Parentheses                                           |
| **       | Exponentiation                                        |
| +x -x ~x | Unary plus, unary minus, and bitwise NOT              |
| * / // % | Multiplication, division, floor division, and modulus |
| + -      | Addition and subtraction                              |
| << >>    | Bitwise left and right shifts                         |
| &        | Bitwise AND                                           |
| ^        | Bitwise XOR                                           |
|          | Bitwise OR                                            |